
Document number	P2472R1
Date	2022-01-31
Reply-to	Jarrad J. Waterloo < descender76@gmail.com >
Audience	Library Evolution Working Group (LEWG)

make `function_ref` more functional

Table of contents

- [make `function\_ref` more functional](#)
 - [Abstract](#)
 - [Motivating examples](#)
 - [Solution](#)
 - [Feature test macro](#)
 - [Other Languages](#)
 - [Example implementation](#)
 - [Acknowledgments](#)
 - [References](#)

Abstract

This document proposes adding additional constructors to `function_ref` ¹ in order to make it easier to use, more efficient and safer to use with common use cases.

Currently, a `function_ref`, can be constructed from a lambda/functor, a free function pointer and a member function pointer. While the lambda/functor use case does support type erasing the `this` pointer, its free/member function pointer constructors does NOT allow type erasing any arguments, even though these two use cases are common.

`function_ref`

	Stateless	Stateful
Lambda/Functor	✓	✓

Free Function	✓	✗
Member Function	✓	✗

Motivating Examples

Given

```

struct cat {
    void walk() {
    }
};

void leap(cat& c) {
}

void catwalk(cat& c) {
    c.walk();
}

struct callback {
    cat* c;
    void (*f)(cat&);
};

cat c;

```

member/free function with type erasure

Since typed erased free and member functions are not currently supported, the current `function_ref` proposal forces its users to create an unnecessary temporary functor likely with `std::bind_front` or a stateful/capturing lambda which the developer hopes the optimizer elides.

member function with type erasure

C/C++ core language	<pre> callback cb = {&c, [](cat& c){c.walk();}}; // or callback cb = {&c, catwalk}; </pre>
---------------------	--

function_ref	<pre> // separate temp needed to prevent dangling // when temp is passed to multiple arguments auto temp = [&c]() {c.walk();}; function_ref<void()> fr = temp; // or when given directly as a function argument some_function([&c]() {c.walk();}); </pre>
proposed	<pre> function_ref<void()> fr = {nontype<&cat::walk>, c}; </pre>

free function with type erasure

C/C++ core language	<pre> callback cb = [&c, [] (cat& c){leap(c);}]; // or callback cb = [&c, leap]; </pre>
function_ref	<pre> // separate temp needed to prevent dangling // when temp is passed to multiple arguments auto temp = [&c]() {leap(c);}; function_ref<void()> fr = temp; // or when given directly as a function argument some_function([&c]() {leap(c);}); </pre>
proposed	<pre> function_ref<void()> fr = {nontype<leap>, c} </pre>

This has numerous disadvantages when compared to what can currently be performed in the C/C++ core language. It is not easy to use, it is inefficient and at times unsafe to use.

Not easy to use

- Unlike the consistent direct initialization of the C/C++ core language example, the initialization of `function_ref` is bifurcated among passing it directly as a function argument or using 2 step initialization by first creating a named temporary. Direct initialization of `function_ref` to a variable would be welcomed if the same `function_ref` is passed to multiple arguments on either the same but more likely different functions. This leads to immediate dangling which must be resolved with an additional line of code. Getting in the habit of only passing `function_ref` as a argument to a function results in users duplicating lambdas when needed to be used

more than once, again needless increasing the volume of code. In both the C/C++ core language and the proposed revision, initialization is consistently the same and safe regardless of whether `function_ref` is first assigned to a variable.

- Why should a stateful lambda even be needed when the signature is compatible?
 - The C/C++ core language example works with a stateless lambda. By requiring a stateful lambda, a new anonymous type gets created which has state. That state's lifetime also needs to be managed by the user.
 - When using a lambda, the user must manually forward the function arguments. In the C/C++ core language example, the syntax can get even simpler when the function already exists because in that case the function pointer is passed to the callback struct because the function signature is compatible.

Inefficient

- By requiring a stateful lambda, a new type is created with a lifetime that must be managed. `function_ref` is a reference to a stateful lambda which also is a reference. Even if the optimizer can remove the cost, the user is still left with the burden in the code.
- In the proposed, nontype does not take a pointer but rather a [member] function pointer initialization statement thus giving the compiler more information to resolve the function selection at compile time rather than run time, granted it is more likely with free functions instead of member functions.

Unsafe

- Direct initialization of `function_ref` outside of a function argument immediately dangles. Some would say that this is no different than other standardized types such as `string_view`. However, this is not true.

C/C++ core language	<pre>std::string_view sv = "hello world"s; // <i>immediately dangling</i> auto c = "hello world"s; std::string_view sv = c; // DOES NOT DANGLE</pre>
function_ref	<pre>function_ref<void()> fr = [&c]() { c.walk(); }; // <i>immediately dangling</i> // or function_ref<void()> fr = [&c]() { leap(c); }; // <i>immediately dangling</i></pre>

```

function_ref<void()> fr = {nontype<&cat::walk>,
                          c}; // DOES NOT DANGLE
proposed // or
function_ref<void()> fr = {nontype<leap>, c} //
                          DOES NOT DANGLE

```

While both the original `function_ref` proposal and the proposed addendum perform the same desired task, the former dangles and the later doesn't. It is clear from the immediately dangling `string_view` example that it dangles because `sv` is a reference and `""s` is a temporary. However, it is less clear in the original `function_ref` example. While it is true and clear that `fr` is a reference and the stateful lambda is a temporary. It is not what the user of `function_ref` is intending or wanting to express. `c` is the state that the user wants to type erase and `function_ref` would not dangle if `c` was the state since it is not a temporary and has already been constructed higher up in the call stack. Further, the member function `walk` or the free function `leap` should not dangle since functions are stateless and also global. So member/free function with type erasure use cases are more like `string_view` when the referenced object is safely constructed.

	easier to use	more efficient	safer to use
C/C++ core language	✓	✓	✓
function_ref	✗	✗	✗
proposed	✓	✓	✓

What is more, member/free function with type erasure are common use cases! member function with type erasure is used by delegates/events in object oriented programming languages and free function with type erasure are common with callbacks in procedural/functional programming languages.

member function without type erasure

The fact that users do not expect stateless things to dangle becomes even more apparent with the member function without type erasure use case.

C/C++ core language	<code>void (cat::*mf)() = &cat::walk;</code>
function_ref	<pre> // separate temp needed to prevent dangling // when temp is passed to multiple arguments auto temp = &cat::walk; </pre>

```
function_ref<void(cat&)> fr = temp;
// or when given directly as a function argument
some_function(&cat::walk);
```

```
proposed      function_ref<void(cat&)> fr =
               {nontype<&cat::walk>};
```

Current `function_ref` implementations store a reference to the member function pointer as the state inside `function_ref`. A trampoline function is required regardless. However the user expected behavior is for `function_ref` referenced state to be unused/nullptr, as ALL of the arguments has to be forwarded since NONE are being type erased. As such dangling is NEVER expected and yet the current `function_ref` [proposal/implementation] does. Similarly, this use case suffers, just as the previous two did, with respect to ease of use, efficiency and safety due to the superfluous lambda/functor and two step initialization. Further if the size of `function_ref` is increased beyond 2 pointers to just make the original proposal work for member function pointers, when it is not needed since only a reference to state and a pointer to a trampoline function is needed, then all use cases are pessimistically increased in size.

	easier to use	more efficient	safer to use
C/C++ core language	✓	✓	✓
function_ref	✗	✗	✗
proposed	✓	✓	✓

free function without type erasure

The C/C++ core language, `function_ref` and the proposed examples are approximately equal with respect to ease of use, efficiency and safety for the free function without type erasure use case. While the proposed nontype example is slightly more wordy because of using the template nontype, it is more consistent with the other three use cases, making it more teachable and usable since the user does not have to know when to do one versus the other i.e. less bifurcation. Also the expectation of unused state and the function being selected at compile time still applies here, as it does for member function without type erasure use case.

```
C/C++ core language  void (*f)(cat&) = leap;
```

```
function_ref      function_ref<void(cat&)> fr = leap;
```

```
proposed          function_ref<void(cat&)> fr = {nontype<leap>};
```

	easier to use	more efficient	safer to use
C/C++ core language	✓	✓	✓
function_ref	✓	✓	✓
proposed	✓	✓	✓

Remove existing free function constructor?

With the overlap in functionality with the free function without type erasure use case, should the existing free function constructor be removed? NO. Initializing a `function_ref` from a function pointer instead of function pointer initialization statement is still usable in the most runtime of libraries such as runtime dynamic library linking where a function pointer is looked up by a string or some other identifier. It is just not the general case, where users work with declarations found in header files and modules.

Solution

```
template<class R, class... ArgTypes> class function_ref<R(ArgTypes...)
    cv noexcept(noex)>
public:
    // MFP is a member function pointer initialization statement
    // I is an instance of the type that house the member function
    // pointed to by MFP
    template<auto MFP, class I> function_ref(nontype<MFP>, I*) noexcept;

    // MFP is a member function pointer initialization statement
    template<auto MFP> function_ref(nontype<MFP>) noexcept;

    // FP is a free function pointer initialization statement
    // FST is the type of the first parameter of the free function
    // pointed to by FP
    template<auto FP, class FST> function_ref(nontype<FP>, FST*)
        noexcept;

    // FP is a free function pointer initialization statement
    template<auto FP> function_ref(nontype<FP>) noexcept;
};
```

Feature test macro

We do not need a feature macro, because we intend for this paper to modify `std::function_ref` before it ships.

Other Languages

C# and the .NET family of languages provide this via delegates ².

```
// C#
delegate void some_name();
some_name fr = leap; // the stateless free function use case
some_name fr = c.walk; // the stateful member function use case
```

Borland C++ now embarcadero provide this via `__closure` ³.

```
// Borland C++, embarcadero __closure
void(__closure * fr)();
fr = leap; // the stateless free function use case
fr = c.walk; // the stateful member function use case
```

Since `nontype function_ref` handles all 4 stateless/stateful free/member use cases, it is more feature rich than either of the above.

Example implementation

The most up-to-date implementation, created by Zhihao Yuan, is available on Github ⁴

Acknowledgments

Thanks to Arthur O'Dwyer, Tomasz Kamiński, Corentin Jabot and Zhihao Yuan for providing very valuable feedback on this proposal.

References

1. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r6.html>↔

2. <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/using-delegates>↵
3. <http://docwiki.embarcadero.com/RADStudio/Sydney/en/Closure>↵
4. https://github.com/zhihaoy/nontype_functional/blob/main/include/std23/function_ref.h↵